

The Four Golden Signals of AI Coding Agents

CONF42

Applying twenty years of SRE thinking to the production system running on your laptop.

WHO IS TALKING

Elliot Gardiner

Seventeen years of infrastructure, SRE and AI platform engineering — now instrumenting the agents.

 Code Telemetry

AI Coding Observability
Founder · 2026 → now · codetelemetry.com

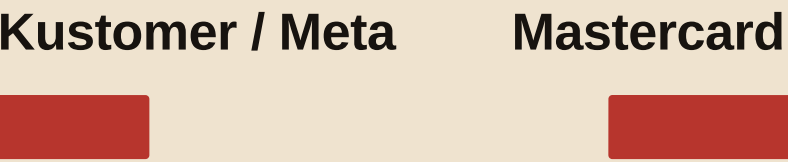
 Mirror Panel

Synthetic Panel Research
Founder · 2026 → now · mirrorpanel.io

HOW I GOT HERE

2009 → 2026

AI engineering



SRE & platform



Infra & app engineering



2009 2016 2020 2026 · Code Telemetry

WHERE THIS COMES FROM

The original model

If you can measure only four things about a running system, measure these.

01 Latency How long the work takes	02 Traffic How much work arrives	03 Errors How much of it fails	04 Saturation How close to the limit
---	---	---	---

THE PREMISE

Your coding agent is a production system .

It runs unattended, for a long time, against your real repository.

Why the framing holds



Calls external services

model APIs, MCP servers,
shells, test runners



Consumes constrained resources

tokens, context window, rate
limit, budget



Retries failures

and sometimes retries the same
failure



Maintains state

a context window that degrades
as it fills



Produces consequential output

commits, migrations, deploy
config

**Five properties we already know
how to instrument — everywhere
except here.**

THE GAP

The operator's blind spot

A SERVICE YOU OPERATE		AN AGENT YOU LAUNCHED	
latency p99	214 ms	elapsed	47:12
traffic	1,240 rps	tool failures	—
error rate	0.04 %	tests run	—
saturation	61 % cpu	context pressure	—

Claude has been working for 47 minutes. Is it working?

THE TRANSLATION

The golden signals, translated

same four questions, new system

SIGNAL	IN PRODUCTION	IN AN AGENT SESSION
Traffic	requests, RPS, queue depth	prompts, tool calls, tokens, edits
Errors	5xx, exceptions, timeouts	reported tool failures, failed tests, reverted edits
Latency	request duration, p99	time to first edit, to green tests, to reviewable change
Saturation	CPU, memory, connections	context pressure, rate limits, concurrency

Status is not correctness

IN A SERVICE

4xx → **error?**

The request failed, but the service may have behaved exactly as specified.

IN AN AGENT SESSION

FAILED → **error?**

The process returned nonzero, but the agent may have learned exactly what it needed.

And 200 or exit 0 can still produce the **wrong outcome.**

THE FIFTH QUESTION: WAS THE RESULT CORRECT?

Google SRE: "All systems should care about correctness."

Seven failed calls. How many were actually errors?

| ELL-51 – Bring task-size API latency within its two-second target.

FAILED TOOL CALLS ONLY

Seven Bash calls, all in the first ten minutes.



9:43:04	5.5s	repo layout probe
9:44:16	5.9s	repo layout probe
9:48:15	7.3s	ls .github/workflows
9:48:38	6.7s	environment probe
9:49:03	7.2s	environment probe
9:52:23	6.0s	ls -d node_modules && npx ts-node
9:52:31	5.9s	ls -d node_modules ×2 →

ONE OF THE SEVEN, READ CLOSELY

Bash **FAILED** 5.9s · exit 2

```
$ ls -d node_modules packages/web/node_modules → No such file or directory (×2)
```

For this probe, the non-zero exit **is the answer—not a fault.**

9:52:39 PM · npm ci **OK** – eight seconds later

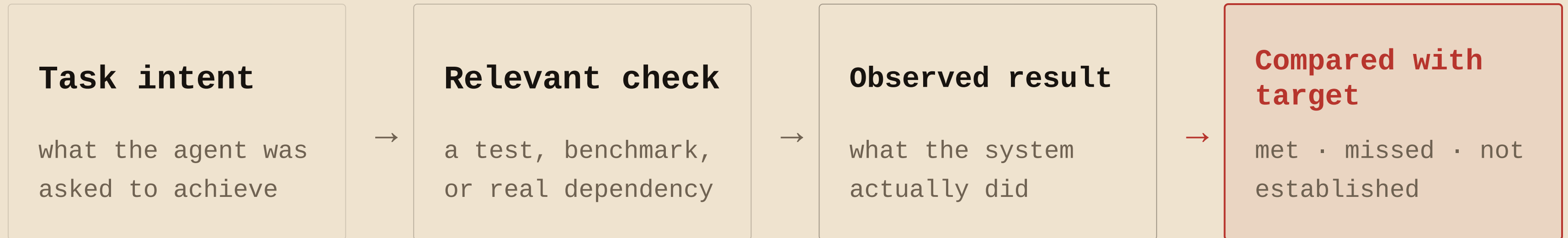
THE TASK'S SUCCESS CRITERION

TARGET	≤ 2 seconds	p95, warm
OBSERVED	– never measured	
VERDICT	Met · Missed · Not established	

“Needs a benchmark against real data, which I have not run.”

Tool status is evidence, not a correctness verdict. **Correctness is judged against the requested outcome.**

How correctness is established



Validation is the evidence behind a **correctness claim**.

Failure patterns in agent sessions

Retry loop

test ✗ → edit → test ✗ → edit → test ✗

Repeated edits

auth.ts edited ×7 · no test run between

Tool fallback

mcp.search ✗ → bash grep -r

Spend without progress

312k tokens · 2 files changed · 0 tests

Review rework

agent 40 min → human 90 min

ONE OPTIONAL DIAGNOSTIC

Are retries producing evidence?

Not all retries are failures. The useful distinction is whether the next pass teaches us anything.

CONVERGING

test ✗ → edit → different failure ✗ → edit → test ✓

new evidence appears at each pass

STUCK

test ✗ → edit → same failure ✗ → edit → same failure ✗

the same failure repeats without producing new evidence

Convergence is a way to interpret repeated work—not another Golden Signal.

What's the runbook?

01

Environment bootstraps

Modify startup images and scripts to reduce predictable tool failures.

02

Model defaults

Change the default model and reasoning mode for the task classes that need it.

03

agent.md + skills.md

Turn recurring failure patterns into durable instructions and reusable skills.

04

Planning + specs

Resize work and reorganize spec-driven development around reliable agent boundaries.

The dashboard should change the next run - not only explain the last one.

CLOSING

We didn't make production reliable by making software stop failing.

We made failure observable.

Coding agents are becoming part of our production engineering systems. It is time we learned how to operate them.